



Pinia + Autenticação Google (Firebase) no Vuetify

Roteiro de aula — estado global de verdade e login social em um app Vue 3

Disciplina:	Frameworks Modernos para Desenvolvimento de Sistemas (FACET-SNP-310)
Professor:	Ivan Luiz Pedroso Pires
Curso/Semestre:	Sistemas de Informação — UNEMAT (Sinop) — 2026/1
Data:	03/06/2026 (quarta-feira)
Duração estimada:	~2 horas (aula prática ao vivo)
Pré-requisitos:	Vue 3 (Composition API), Vuetify, Vue Router e CRUD em memória (aulas de 13/05, 20/05 e 27/05)
Projeto de apoio:	pasta <code>03-06-2026/</code> (exemplo pronto, roda com <code>bun install && bun dev</code>)

0. Mapa da aula

O objetivo de hoje é dar dois saltos de uma vez, costurados pelo Vuetify:

- **Pinia** — trocar aquele `store/itens.js` caseiro da aula passada por uma store de verdade, oficial do ecossistema Vue.
- **Firestore Authentication** — colocar um **login com a conta Google** funcionando, sem escrever um único endpoint de backend.
- **Rota protegida** — uma página (`/perfil`) que só abre para quem está logado.

Tempo	Bloco	O que fazemos ao vivo
0:00-0:15	Recap + motivação	Lembrar o store manual da aula passada e mostrar onde ele dói. Apresentar o Pinia.
0:15-0:40	Pinia	Instalar, registrar, e escrever a 1ª store (estado, getters, actions).
0:40-1:00	Firebase Console	Criar o projeto, ativar o login Google, copiar a config.
1:00-1:30	Login funcionando	<code>firebase.js</code> + store de auth + <code>UserMenu</code> . Clicar e logar com o Google.
1:30-1:50	Rota protegida	Guarda de rota, <code>storeToRefs</code> , página de perfil. Testar o F5.
1:50-2:00	Erros & exercício	Troubleshooting dos erros clássicos. Passar o exercício.

1. Por que Pinia? (gancho com a aula passada)

Na aula de 27/05 a gente compartilhou a lista entre as rotas com um arquivo solto:

`src/store/itens.js` (a gambiarra que funcionava)

```
import { ref } from 'vue'
export const itens = ref([]) // estado solto
export function salvar(item) { /* ... */ }
export function excluir(id) { /* ... */ }
```

Funciona, mas vai escalando mal: estado, regras e "ações" ficam espalhados, sem organização, sem ferramentas de debug, e cada um inventa um padrão diferente. **Pinia** é a forma *oficial* de fazer isso no Vue — o mesmo conceito (um estado que vive fora dos componentes), porém organizado em *stores* com **estado**, **getters** e **actions**.

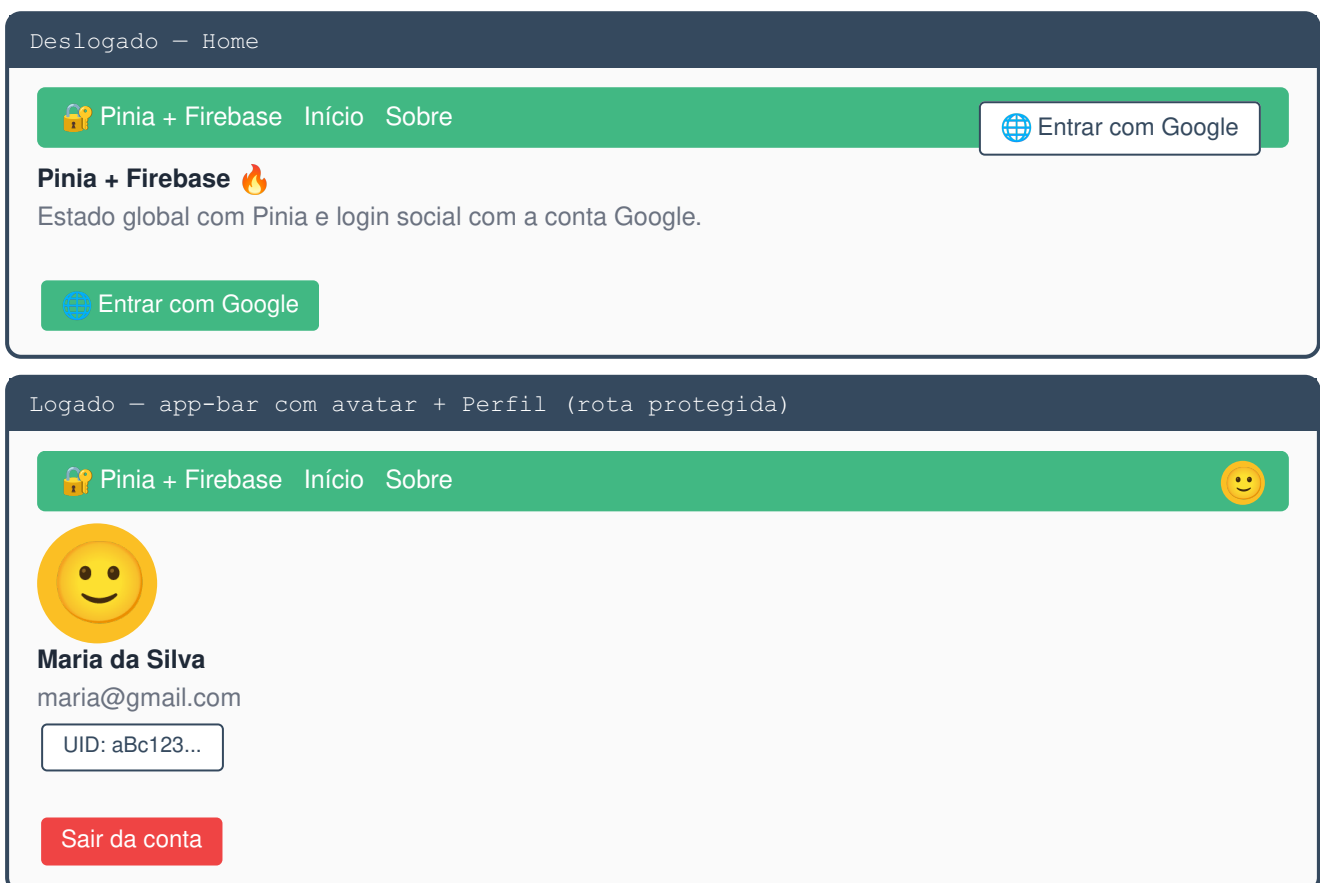
Analogia: uma store Pinia é como um `<script setup>` que não pertence a nenhum componente e é compartilhado por todos. `ref()` vira o estado, `computed()` vira getter, e `function` vira action.

2. Os 3 conceitos do Pinia

Conceito	No Vue puro é...	Para que serve
state	<code>ref()</code> / <code>reactive()</code>	Os dados. Ex.: <code>user</code> , <code>carregando</code> .
getters	<code>computed()</code>	Valores derivados. Ex.: <code>estaLogado</code> = "tem user?".
actions	<code>function</code> (pode ser <code>async</code>)	O que muda o estado. Ex.: <code>entrarComGoogle()</code> .

Usamos o estilo **"setup store"** (uma função que retorna tudo) porque é idêntico ao que já sabemos de `<script setup>`. Existe também o estilo "options" (objeto), mas vamos de setup.

3. O que vamos construir



4. Passo a passo (a aula ao vivo)

1 Criar o projeto e instalar Pinia + Firebase

~5 min

Crie um projeto Vuetify novo (igual às aulas anteriores) e adicione as duas bibliotecas de hoje:

```
# cria o projeto base (Vue 3 + Vite + Vuetify) - escolha "Recommended" + bun
bun create vuetify app-pinia-firebase
cd app-pinia-firebase

# as duas estrelas de hoje:
bun add pinia firebase
```

Atalho: a pasta `03-06-2026/` já tem tudo pronto. Dá para abrir ela e só rodar `bun install`. Mas na aula vale construir do zero para fixar.

2 Ligar o Pinia no app

~3 min

O Pinia precisa ser registrado no app — e **antes do router**, porque o guarda de rota vai usar a store.

`src/plugins/index.js`

```
import { createPinia } from 'pinia'
import vuetify from './vuetify'
import router from '@/router'

export function registerPlugins (app) {
  app.use(createPinia()) // 1º: estado global
  app.use(vuetify)       // 2º: UI
  app.use(router)        // 3º: rotas (usam a store!)
}
```

Ordem importa: se o router subir antes do Pinia, o guarda de rota chama `useAuthStore()` sem um Pinia ativo e o app quebra com `"getActivePinia() was called but there was no active Pinia"`.

3 Criar o projeto no Firebase Console

~12 min

Tudo no navegador, sem código. Vá em `console.firebase.google.com`:

1. **Adicionar projeto** → dê um nome (ex.: *fmds-aula*) → pode desligar o Google Analytics → Criar.
2. No menu lateral: **Build** → **Authentication** → **Get started**.
3. Aba **Sign-in method** → clique em **Google** → **Ativar (Enable)** → escolha um e-mail de suporte → **Salvar**.
4. Agora registre um app Web: ícone de engrenagem `⚙ Configurações do projeto` → role até **Seus apps** → clique no ícone **Web** `</>`.
5. Dê um apelido ao app → **Registrar app**. O Firebase mostra um objeto `firebaseConfig` — **copie esses valores** (vamos usar já já).
6. Confira os domínios liberados: **Authentication** → **Settings** → **Authorized domains**. O `localhost` já costuma vir na lista — se não estiver, adicione.

Erro clássico nº 1: esquecer de ativar o provedor Google (passo 3). Sem isso, o login abre o popup e morre com `auth/operation-not-allowed`.

4 Guardar a config no .env e inicializar o Firebase

~6 min

Crie um arquivo `.env` na raiz com os valores que você copiou (prefixo `VITE_` é obrigatório para o Vite expor a variável):

`.env`

```
VITE_FIREBASE_API_KEY=AIzaSy...suachave
VITE_FIREBASE_AUTH_DOMAIN=fmds-aula.firebaseio.com
VITE_FIREBASE_PROJECT_ID=fmds-aula
VITE_FIREBASE_APP_ID=1:123...:web:abc...
```

Agora um arquivo que inicializa o Firebase **uma única vez** e exporta o que precisamos:

`src/firebase.js`

```
import { initializeApp } from 'firebase/app'
import { getAuth, GoogleAuthProvider } from 'firebase/auth'

const firebaseConfig = {
  apiKey: import.meta.env.VITE_FIREBASE_API_KEY,
  authDomain: import.meta.env.VITE_FIREBASE_AUTH_DOMAIN,
  projectId: import.meta.env.VITE_FIREBASE_PROJECT_ID,
  appId: import.meta.env.VITE_FIREBASE_APP_ID,
}

const app = initializeApp(firebaseConfig)

export const auth = getAuth(app)
export const googleProvider = new GoogleAuthProvider()
```

Sobre "segredo": essas chaves Web não são secretas — são identificadores públicos do projeto (vão no bundle do navegador de qualquer forma). A segurança vem dos *domínios autorizados* e das *regras*. Usamos `.env` por organização e por hábito profissional. Lembre de pôr `.env` no `.gitignore`.

5 A store de autenticação (o coração da aula)

~12 min

Aqui o Pinia e o Firebase se encontram. A store guarda o `user`, expõe getters prontos e tem as actions de entrar/sair. O segredo é o `onAuthStateChanged`: o Firebase **nos avisa sozinho** sempre que o login muda (inclusive ao recarregar a página).

src/stores/auth.js

```
import { defineStore } from 'pinia'
import { ref, computed } from 'vue'
import { signInWithPopup, signOut, onAuthStateChanged } from 'firebase/auth'
import { auth, googleProvider } from '@/firebase'

export const useAuthStore = defineStore('auth', () => {
  // STATE
  const user = ref(null) // usuário do Firebase, ou null
  const carregando = ref(true) // true até o Firebase responder
  const erro = ref(null)

  // GETTERS (computed = sempre reativo)
  const estaLogado = computed(() => user.value !== null)
  const nome = computed(() => user.value?.displayName ?? 'Visitante')
  const email = computed(() => user.value?.email ?? '')
  const foto = computed(() => user.value?.photoURL ?? '')

  // Promise que resolve quando o Firebase responder pela 1ª vez.
  // O guarda de rota espera por ela (resolve o problema do F5).
  let resolverPronto
  const pronto = new Promise((r) => { resolverPronto = r })

  // O Firebase chama isto sozinho: ao entrar, ao sair e ao recarregar.
  onAuthStateChanged(auth, (u) => {
    user.value = u
    carregando.value = false
    resolverPronto()
  })

  // ACTIONS
  async function entrarComGoogle () {
    erro.value = null
    try {
      await signInWithPopup(auth, googleProvider)
      // não setamos user aqui: o onAuthStateChanged faz isso
    } catch (e) {
      erro.value = e.message
    }
  }

  async function sair () {
    await signOut(auth)
  }

  return { user, carregando, erro, estaLogado, nome, email, foto, pronto, entrarComGoogle, sair }
})
```

Ideia-chave: ninguém "seta o usuário na mão". A action só dispara o popup; quem atualiza o estado é o `onAuthStateChanged`. Uma fonte da verdade só.

6 O componente de login no app-bar

~8 min

Um componente que mostra "Entrar com Google" quando deslogado, e o avatar + menu quando logado.

src/components/UserMenu.vue

```
<template>
  <!-- ainda carregando? mostra spinner -->
  <v-progress-circular v-if="auth.carregando" indeterminate size="24" />

  <!-- deslogado: botão de entrar -->
  <v-btn v-else-if="!auth.estaLogado" prepend-icon="mdi-google"
    @click="auth.entrarComGoogle">Entrar com Google</v-btn>

  <!-- logado: avatar que abre um menu -->
  <v-menu v-else>
    <template #activator="{ props }">
      <v-btn v-bind="props" icon>
        <v-avatar size="36"><v-img :src="auth.foto" /></v-avatar>
      </v-btn>
    </template>
    <v-list>
      <v-list-item :title="auth.nome" :subtitle="auth.email" />
      <v-divider />
      <v-list-item to="/perfil" title="Meu perfil" prepend-icon="mdi-account" />
      <v-list-item title="Sair" prepend-icon="mdi-logout" @click="auth.sair" />
    </v-list>
  </v-menu>
</template>

<script setup>
import { useAuthStore } from '@stores/auth'
const auth = useAuthStore() // auth.estaLogado/nome no template são reativos
</script>
```

E no `App.vue`, basta pôr o componente na `v-app-bar`:

src/App.vue (trecho)

```
<v-app-bar color="primary">
  <v-app-bar-title>👤 Pinia + Firebase</v-app-bar-title>
  <v-btn to="/" variant="text">Início</v-btn>
  <v-spacer />
  <UserMenu />
</v-app-bar>
```

Momento "uau" da aula: rode `bun dev`, clique em *Entrar com Google*, escolha a conta — e o avatar aparece no canto. Login social sem backend nenhum. 🎉

7 Proteger a rota /perfil (guarda de navegação)

~10 min

Marcamos a rota como "só para logados" com `meta.requiresAuth` e barramos no `beforeEach`.

src/router/index.js

```
import { createRouter, createWebHistory } from 'vue-router'
import { useAuthStore } from '@/stores/auth'

const routes = [
  { path: '/', component: () => import('@/pages/Home.vue') },
  { path: '/perfil', component: () => import('@/pages/Perfil.vue'),
    meta: { requiresAuth: true } }, // <-- só logados
  { path: '/sobre', component: () => import('@/pages/Sobre.vue') },
]

const router = createRouter({ history: createWebHistory(), routes })

router.beforeEach(async (to) => {
  const auth = useAuthStore()
  await auth.pronto // espera o Firebase restaurar a sessão
  if (to.meta.requiresAuth && !auth.estaLogado) {
    return '/' // não logado? volta pra Home
  }
})

export default router
```

Erro clássico nº 2 (o F5): sem o `await auth.pronto`, ao recarregar a página em `/perfil` o guarda roda *antes* de o Firebase restaurar a sessão — então `estaLogado` ainda é `false` e o usuário logado é chutado pra Home. Esperar a Promise resolve isso.

8 A página de perfil + storeToRefs

~6 min

Boa hora para ensinar `storeToRefs`: para desestruturar estado/getters sem perder a reatividade.

src/pages/Perfil.vue (trecho)

```
<script setup>
import { storeToRefs } from 'pinia'
import { useAuthStore } from '@/stores/auth'

const auth = useAuthStore()
// CERTO: mantém a reatividade ao desestruturar
const { user, nome, email, foto } = storeToRefs(auth)

// ERRADO: const { nome } = auth --> vira valor "congelado", não atualiza!
</script>
```

Regra de bolso: *state e getters* → via `storeToRefs`. *actions* (funções) → pode pegar direto da store (`auth.sair`), elas não perdem reatividade.

9 Testar o fluxo completo

~5 min

- Deslogado: a Home mostra "Entrar com Google"; tente abrir `/perfil` na barra de endereço → é redirecionado pra Home. ✓
- Clique em entrar → escolha a conta → avatar aparece, nome/e-mail no menu. ✓
- Abra `/perfil` → agora entra. Dê `F5` → continua logado (sem ser expulso). ✓
- Clique em *Sair* → volta a deslogado, a UI inteira reage sozinha. ✓

5. Troubleshooting — os erros que vão aparecer

Erro no console	Causa provável	Como resolver
<code>auth/operation-not-allowed</code>	Provedor Google não ativado.	Authentication → Sign-in method → ativar Google.
<code>auth/unauthorized-domain</code>	Domínio não autorizado.	Authentication → Settings → Authorized domains → add <code>localhost</code> .
<code>auth/invalid-api-key</code>	<code>.env</code> errado/ausente ou esqueceu de reiniciar o Vite.	Confira o <code>.env</code> e reinicie o <code>bun dev</code> (o Vite só lê o <code>.env</code> no boot).
<code>auth/popup-blocked</code>	Navegador bloqueou o popup.	Liberar pop-ups do <code>localhost</code> e clicar de novo (gesto do usuário).
<code>popup-closed-by-user</code>	Fechou a janela antes de concluir.	Normal — só tentar de novo. Já tratamos no <code>catch</code> .
<code>no active Pinia</code>	Router registrado antes do Pinia.	No <code>plugins/index.js</code> , <code>createPinia()</code> vem primeiro.

6. Cola rápida

Preciso...	Código
Definir uma store	<code>defineStore('nome', () => { ... return {...} })</code>
Usar a store	<code>const s = useNomeStore()</code>
Desestruturar mantendo reatividade	<code>const { x } = storeToRefs(s)</code>
Login com Google	<code>signInWithPopup(auth, googleProvider)</code>
Logout	<code>signOut(auth)</code>
Ouvir login/logout	<code>onAuthStateChanged(auth, (u) => { ... })</code>

7. Exercício — para entregar

Atividade: "Minha área logada" (Pinia + login Google)

Valor: 1,0 ponto (compõe a Nota 2) • **Modalidade:** individual ou em dupla • **Entrega:** SIGAA até **23h59 de 09/06/2026** • **Base:** evolua o projeto da aula de hoje.

Contexto: use o app de hoje (login com Google + Pinia) como ponto de partida e adicione uma **área pessoal protegida** com dados que pertencem ao usuário logado — gerenciados por uma **segunda store Pinia**, sua. Escolha um tema: lista de tarefas, favoritos de filmes, anotações, metas de estudo, lista de compras... (declare o tema no `README`).

Requisitos

- **R1 — Login Google funcionando** (reaproveitado da aula): app-bar com "Entrar"/avatar e "Sair".
- **R2 — Segunda store Pinia** própria (ex.: `stores/itens.js`) com *state*, pelo menos **um getter** (ex.: `total`) e *actions* de **adicionar** e **remover**.
- **R3 — Rota protegida** `/minha-area` (com `meta.requiresAuth` + guarda) que só abre logado e onde o usuário **cadastra e remove** itens do seu tema.
- **R4 — Personalização:** a área mostra o **nome** e a **foto** do usuário (vindos da store de auth) — ex.: *"Tarefas de Maria"*.
- **R5 — Vuetify:** a interface usa pelo menos 4 componentes Vuetify (ex.: `v-text-field` , `v-btn` , `v-list` / `v-card` , `v-icon`) e o tema com cores próprias.
- **R6 — storeToRefs:** ao consumir a store na página, desestruture state/getters com `storeToRefs` (sem quebrar a reatividade).

Desafio extra (opcional, +0,2 na qualidade)

Faça os itens **sobreviverem ao F5** salvando a lista no `localStorage` (dica: um `watch` na store gravando, e leitura na inicialização). Bônus se a lista for separada por usuário (use o `uid` na chave).

Critérios de avaliação

Critério	O que avaliamos	Pontos
Login + rota protegida	Google funcionando e <code>/minha-area</code> barrando visitante (R1, R3).	0,30
Store Pinia própria	State + getter + actions add/remove, bem organizada (R2).	0,30
Personalização do usuário	Nome/foto do logado aparecem na área (R4) + <code>storeToRefs</code> (R6).	0,20
Vuetify & capricho	Componentes Vuetify, tema, sem erros no console (R5).	0,20
TOTAL		1,00

Entrega: zipe o projeto sem a pasta `node_modules` e sem o `.env` (suas chaves ficam fora!). No `README.md` : tema escolhido, integrantes e como rodar. Nome do arquivo: `fmds_pinia_NOME1_NOME2.zip` . Só uma pessoa da dupla envia.

Ordem sugerida: (1) app de hoje rodando → (2) rota `/minha-area` protegida vazia → (3) segunda store com add/remove → (4) formulário + lista lendo da store → (5) nome/foto do usuário → (6) tema e capricho.

Bônus moral — o meme do estado global

 AULA PASSADA: "VOU GUARDAR O ESTADO NUMA VARIÁVEL GLOBAL"	 HOJE: "USO PINIA, SOU UM PROFISSIONAL"
 "IMPLEMENTEI LOGIN, OAUTH, SESSÃO E REFRESH EM 5 MINUTOS" (O FIREBASE FEZ 99% — EU SÓ CLIQUEI EM 'ATIVAR GOOGLE')	
 "POR QUE AO DAR F5 ELE ME DESLOGA?!"	 ESQUECEU O AWAIT AUTH.PRONTO NO GUARDA

"Autenticação é fácil: são só 3 linhas de código e 47 cliques no console." — Provérbio Firebase

Bom código, pessoal! ✦ Projeto-exemplo completo na pasta [03-06-2026/](#) ✦ Exercício no SIGAA até **23h59 de 09/06/2026**.